

Decision Trees

Prof. Artur Jordão

Definition

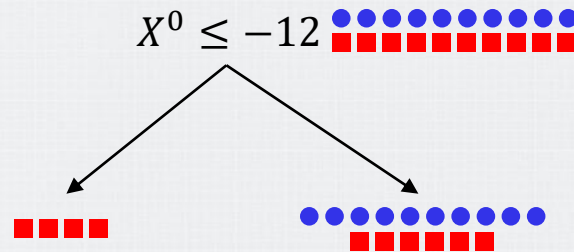
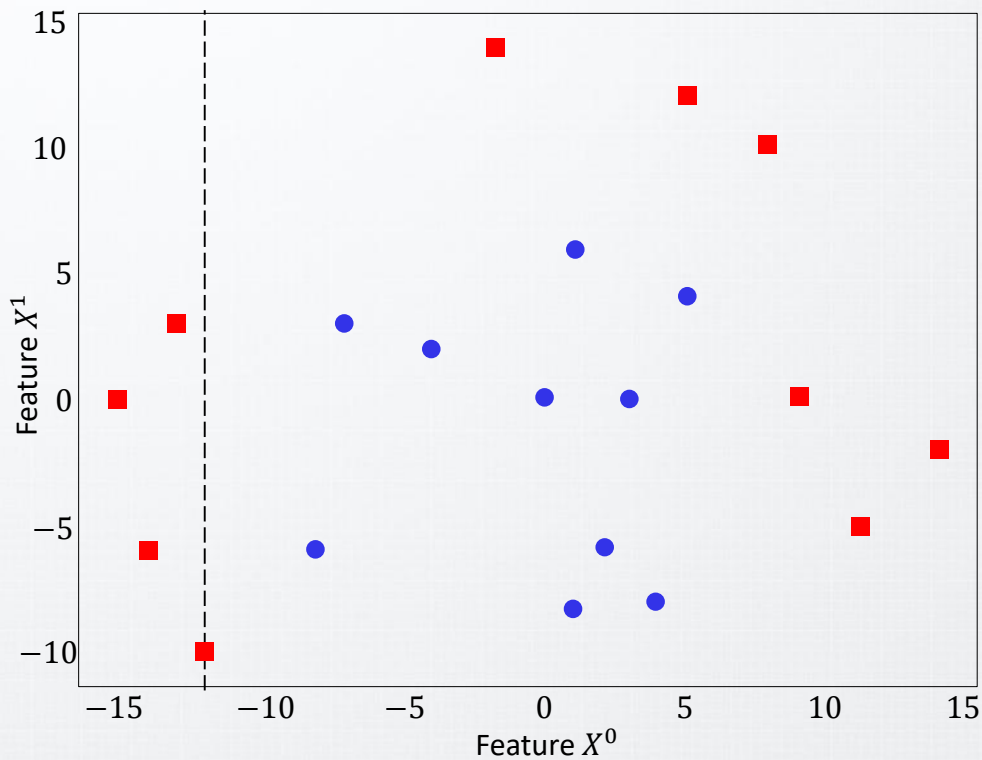
Decision Trees

- A decision tree is a hierarchical data structure implementing the divide-and-conquer strategy
- Nonparametric method for classification and regression
- A hierarchical model of recursive splits (tests) in the following form
 - $\begin{cases} \text{feature } i \leq \tau, \text{ go left} \\ \text{feature } i > \tau, \text{ go right} \end{cases}$

Overview

Decision Trees

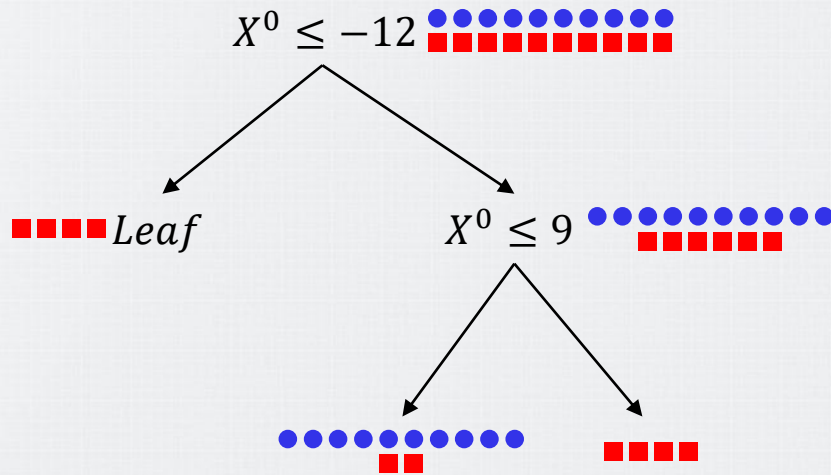
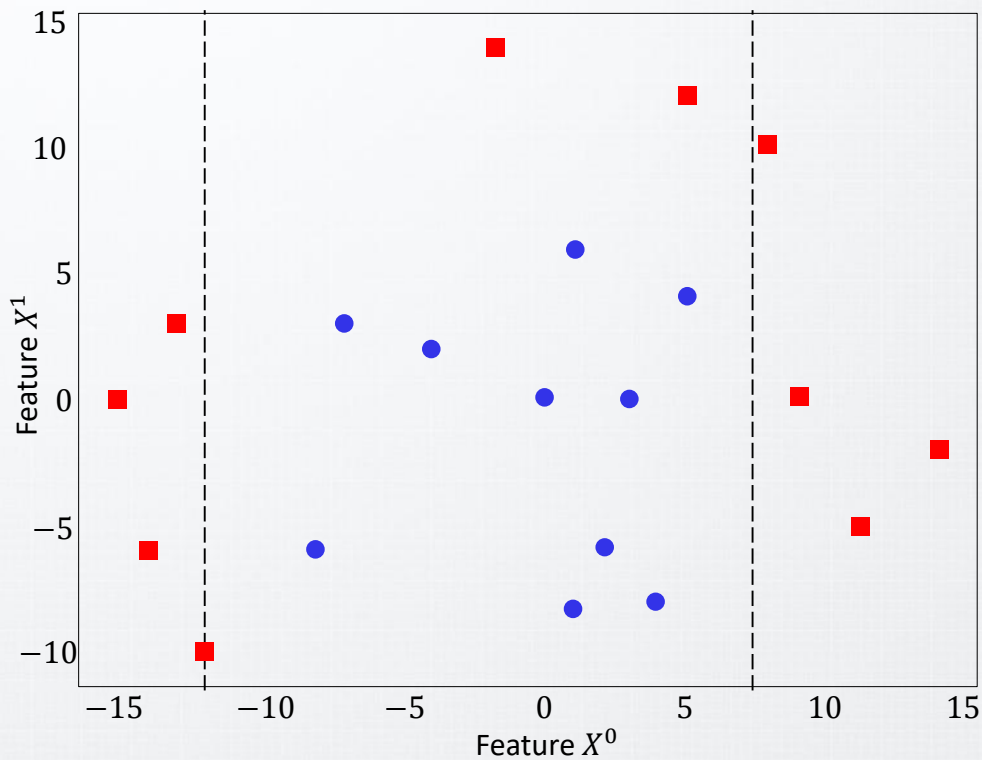
- Step by step



Overview

Decision Trees

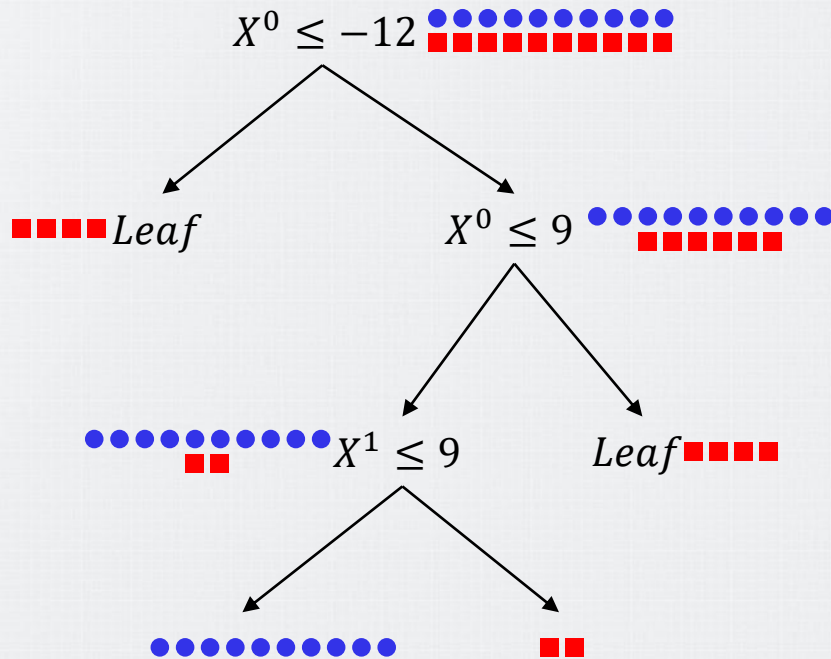
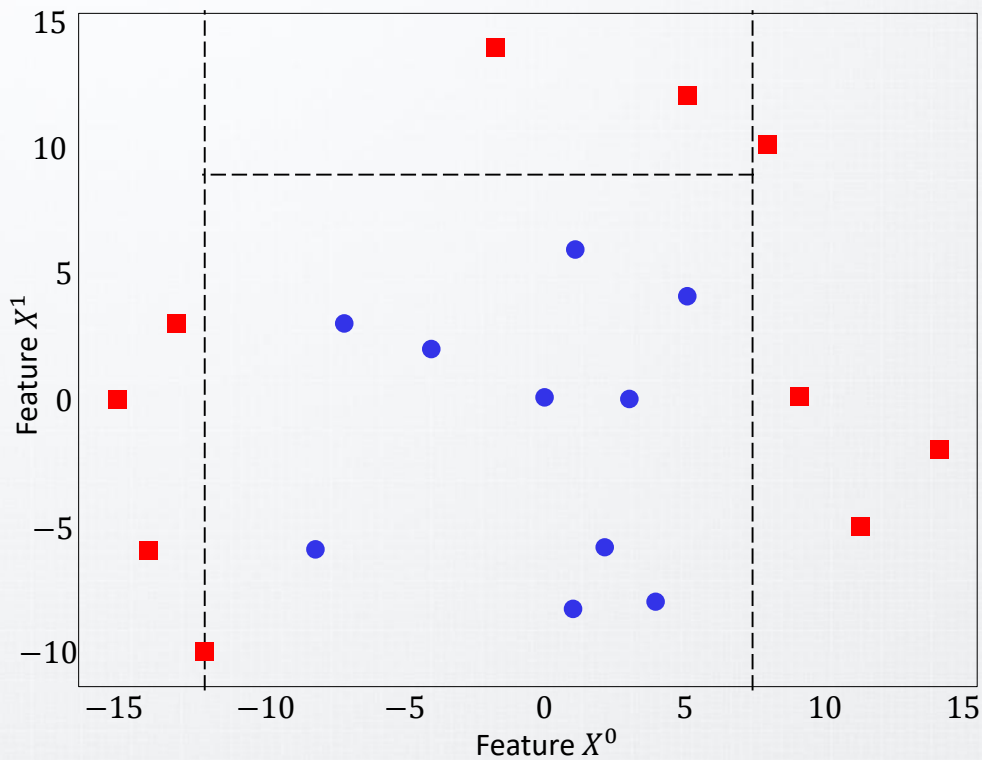
- Step by step



Overview

Decision Trees

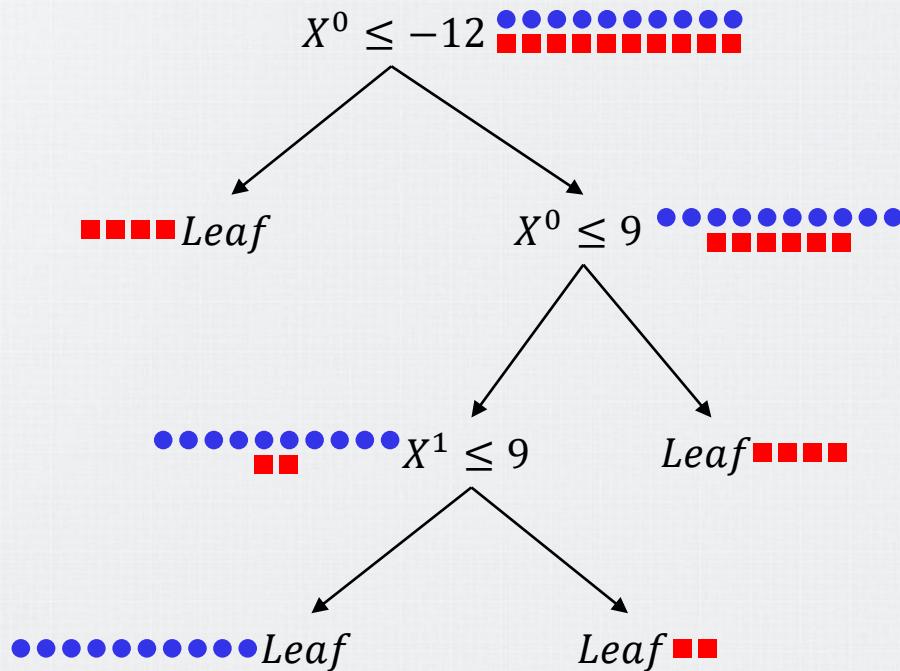
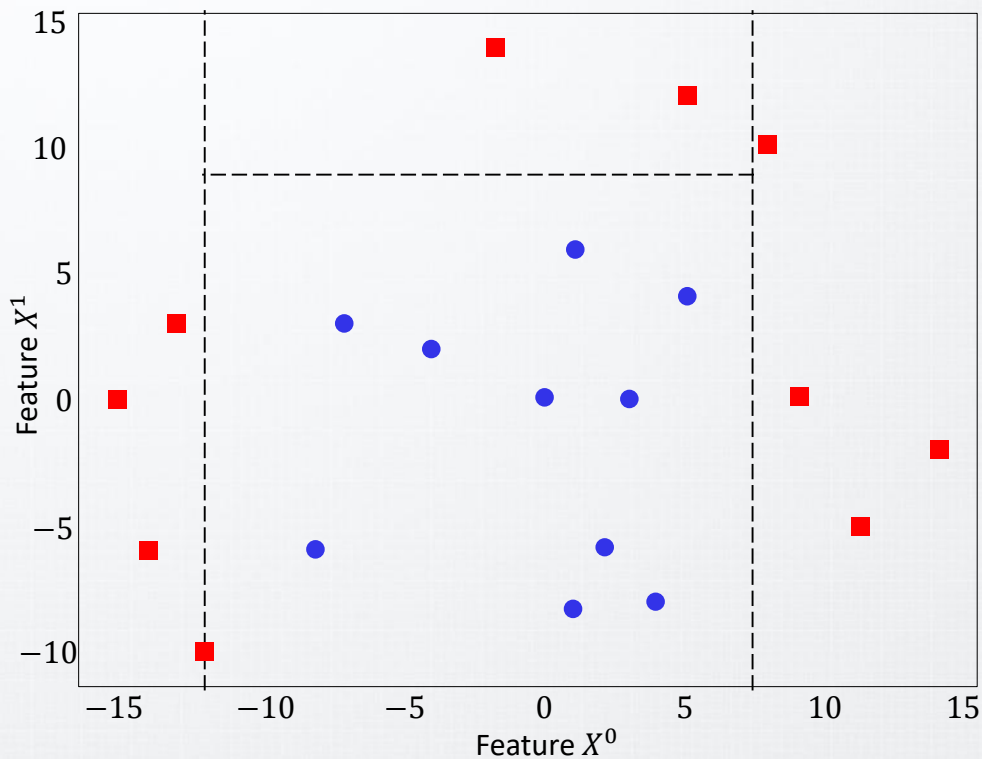
- Step by step



Overview

Decision Trees

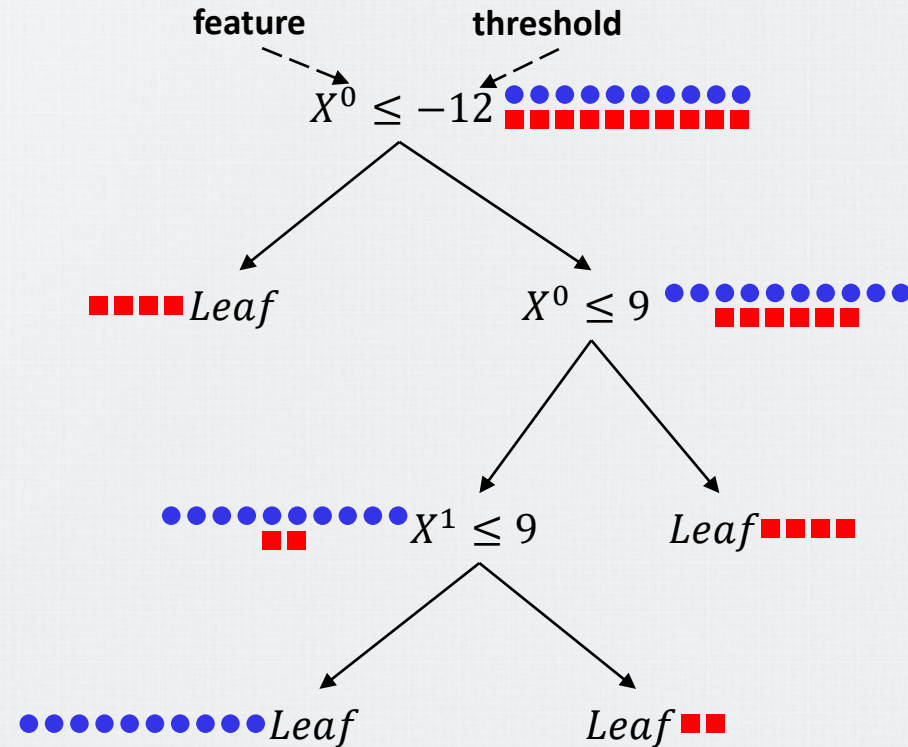
- Step by step



Problem Definition

Decision Trees

- How to find features and thresholds that partition the feature space in a principled manner?



Building a Decision Tree

Impurity Measure

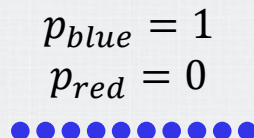
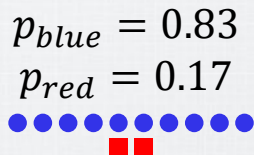
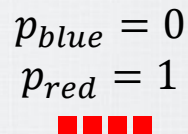
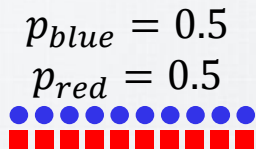
Building a Decision Tree

- The first question in building a Decision Tree is: how to find the decisions (splits) in a node?
- Popular functions to measure impurity
 - Entropy
 - Gini index
 - Misclassification (classification error rate)
- If a node is not pure, it should split the samples to decrease impurity
 - Multiple attributes are available for splits

Impurity Measure

Building a Decision Tree

- The estimate for the probability of class c_i in a node is
 - $p_i = \frac{n_i}{n}$
 - n_i is the number of samples from class i
 - n is the number of samples



Entropy

Building a Decision Tree

- Entropy measures the degree of randomness in data
 - **Lower entropy implies greater predictability**
- Entropy for k classes
 - $Entropy(.) = -\sum_{i=1}^k p_i \log_2(p_i)$

Entropy

Building a Decision Tree

- Impurity with entropy
 - $Entropy(parent) - [\eta * Entropy(child_{left}) + (1 - \eta) * Entropy(child_{right})]$
 - η is the fraction of the samples where $X^i \leq \tau$

$$\begin{aligned}
 & - \left(\frac{0}{4} \log_2 \frac{0}{4} + \frac{4}{4} \log_2 \frac{4}{4} \right) = 0 \quad \left(\frac{10}{20} \log_2 \frac{10}{20} + \frac{10}{20} \log_2 \frac{10}{20} \right) = 1 \\
 & \quad \quad \quad \eta = \frac{16}{20} \\
 & - \left(\frac{10}{16} \log_2 \frac{10}{16} + \frac{6}{16} \log_2 \frac{6}{16} \right) = 0.95
 \end{aligned}$$

[Hands-on](#)



Gini Index

Building a Decision Tree

- The Gini index measures how often a randomly chosen example receives an incorrect label when one assigns labels at random according to the class distribution
 - The Gini index is low when class probabilities (p_i) concentrate near zero or one
- Gini for k classes
 - $Gini(.) = \sum_{i=1}^k p_i(1 - p_i)$

$$\left(\frac{0}{4}\right) * \left(1 - \frac{0}{4}\right) + \left(\frac{4}{4}\right) * \left(1 - \frac{4}{4}\right) = 0$$

$$\left(\frac{10}{20}\right) * \left(\frac{10}{20}\right) + \left(\frac{10}{20}\right) * \left(\frac{10}{20}\right) = 0.5$$

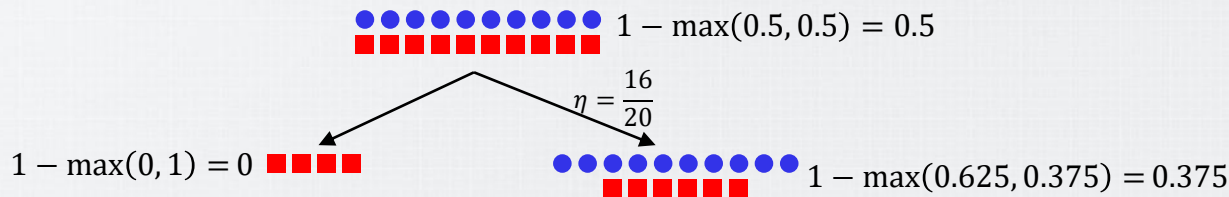
$$\left(\frac{10}{16}\right) * \left(1 - \frac{10}{16}\right) + \left(\frac{6}{16}\right) * \left(1 - \frac{6}{16}\right) = 0.46875$$

$$\eta = \frac{16}{20}$$

Classification Error Rate

Building a Decision Tree

- Since we plan to classify each observation by the most frequent class in its region, a natural impurity measure is the classification error rate
 - The fraction of the training observations in that region that **do not** belong to the most common class
- Classification Error Rate for k classes
 - $1 - \max_k p_i$



Threshold

Building a Decision Tree

- How to define the threshold τ (Split points)?
- Naïve solution
 - $\tau_i \in \{x^i\}_{i=1,\dots,m}$
- Splitting is the most expensive part of real-world decision tree learning applications

[Hands-on](#)



Threshold

Building a Decision Tree

- Efficient methods exist for finding good split points
 - For example, start by sorting the values of the attribute, and then consider only split points that are between two examples in sorted order that have different classifications
- Observe that, since each split considers a single feature at a time, the procedure partitions the feature space along **axis-aligned (orthogonal)** directions

Types of Nodes

Building a Decision Tree

- Internal nodes
 - Execute branching tests
 - $X^i \leq \tau$
- Leaf nodes
 - Specify the value that the node (function) returns
 - Assign a label to the sample (perform classification)
 - $\hat{y} = \underset{k}{\operatorname{argmax}} n_k$
- The tree grows through the internal nodes to reach the leaf nodes
 - From branching tests to predictions

Stopping Criteria

Building a Decision Tree

- To prevent overfitting, we stop the growth (pruning) process according to predefined stopping criteria
- Types of stopping criteria
 - The node reaches a predefined depth
 - The impurity (leaf node) is zero (i.e., the node classifies all examples correctly)
 - The number of samples in the node is less than a predefined threshold
 - The split reduces the impurity less than some ϵ

Algorithm Overview

Building a Decision Tree

```
BuildTree( $X, Y$ , Depth)
```

```
    node = TreeNode()
```

```
    If StopCriterion
```

```
        node.label =  $\hat{y} \triangleleft \underset{k}{\operatorname{argmax}} n_k$ 
```

```
    return node
```

```
    ( $i^*, \tau^*$ ) =  $\underset{i \in \{1, \dots, m\}, \tau \in T}{\operatorname{argmax}} \operatorname{Gain}(X, Y, i, \tau)$ 
```

```
    node. $i^*$  =  $i^*$ , node. $t^*$  =  $t^*$ 
```

```
    node.LeftChild = BuildTree( $\{(x, y) | (x, y) \in X, x^{i^*} \leq t^*\}$ , Depth + 1)
```

```
    node.RightChild = BuildTree( $\{(x, y) | (x, y) \in X, x^{i^*} > t^*\}$ , Depth + 1)
```

```
    return Node
```

Prediction

Building a Decision Tree

- After building the decision tree, we can perform the prediction by

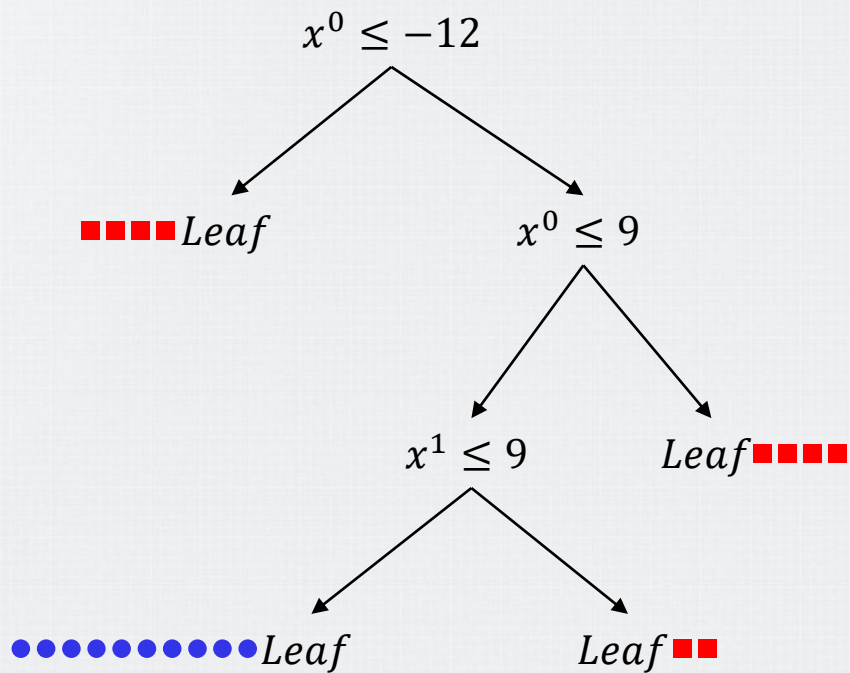
$$\text{predict}(x, \text{root}) = \begin{cases} \text{node.label, node is a leaf} \\ \text{predict}(x, \text{node.LeftChild}), x^{\text{node.i}^*} \leq \text{node}.\tau^* \\ \text{predict}(x, \text{node.RightChild}), \text{Otherwise} \end{cases}$$

Prediction

Building a Decision Tree

- Practical example

$x \in \mathbb{R}^2$	$\hat{y}$
(4, 0)	?
(5, 1)	?
(-15, 4)	?
(3, 12)	?

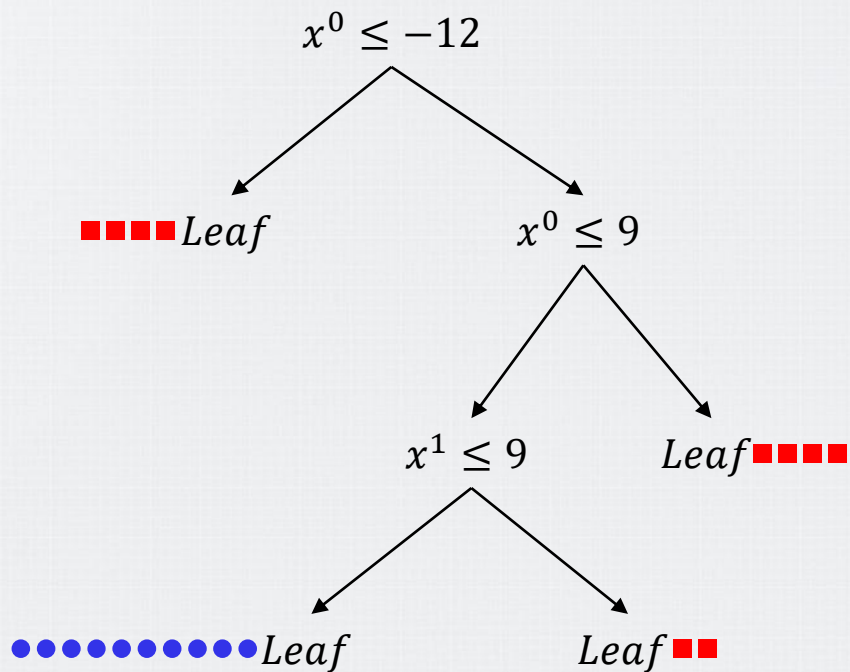


Prediction

Building a Decision Tree

- Practical example

$x \in \mathbb{R}^2$	$\hat{y}$
(4, 0)	blue circle
(5, 1)	blue circle
(-15, 4)	red square
(3, 12)	red square



Algorithm Overview

Building a Decision Tree

```
prediction( $x, node$ )
```

```
  If  $node.leaf$ 
```

```
    return  $\hat{y}$ 
```

```
  If  $x^{node.i^*} \leq node.\tau^*$ 
```

```
    return prediction( $x, node.LeftChild$ )
```

```
  else
```

```
    return prediction( $x, node.RightChild$ )
```

Decision Trees for Regression and Categorical Target Variables

Decision Trees for Regression

Decision Trees for Regression and Categorical Target Variables

- A regression tree follows almost the same procedure as a classification tree
 - The impurity measure for classification gives place to a measure suitable for regression
- Split
 - $R_{left}(i, \tau) = \{(x, y) \in X | X^i \leq \tau\}$, $R_{right}(i, \tau) = \{(x, y) \in X | X^i > \tau\}$
- The node seeks the value of i and τ that minimize
 - $\sum_{x_i \in R_{left}} (y_i - \hat{y}_{R_{left}})^2 + \sum_{x_i \in R_{right}} (y_i - \hat{y}_{R_{right}})^2$
 - $\hat{y}$ denotes the average value
- Leaf node
 - Predict the response for a test sample using the average of the training samples in the node

Decision Trees for Categorical Data

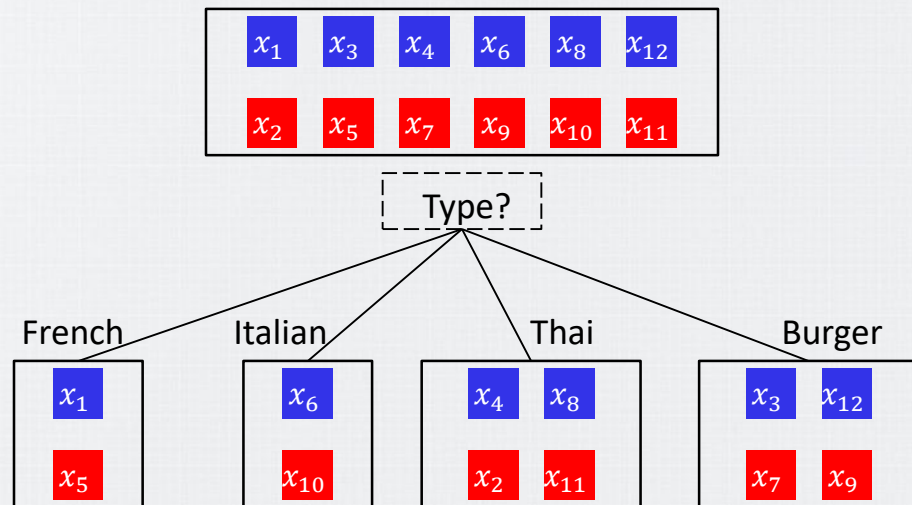
Building a Decision Tree

Sample	Features										y	
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait	
x_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0–10	Yes	
x_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30–60	No	
x_3	No	Yes	No	No	Some	\$	No	No	Burger	0–10	Yes	
x_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10–30	Yes	
x_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No	
x_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0–10	Yes	
x_7	No	Yes	No	No	None	\$	Yes	No	Burger	0–10	No	
x_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0–10	Yes	
x_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No	
x_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10–30	No	
x_{11}	No	No	No	No	None	\$	No	No	Thai	0–10	No	
x_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30–60	Yes	

Decision Trees for Categorical Data

Decision Trees for Regression and Categorical Target Variables

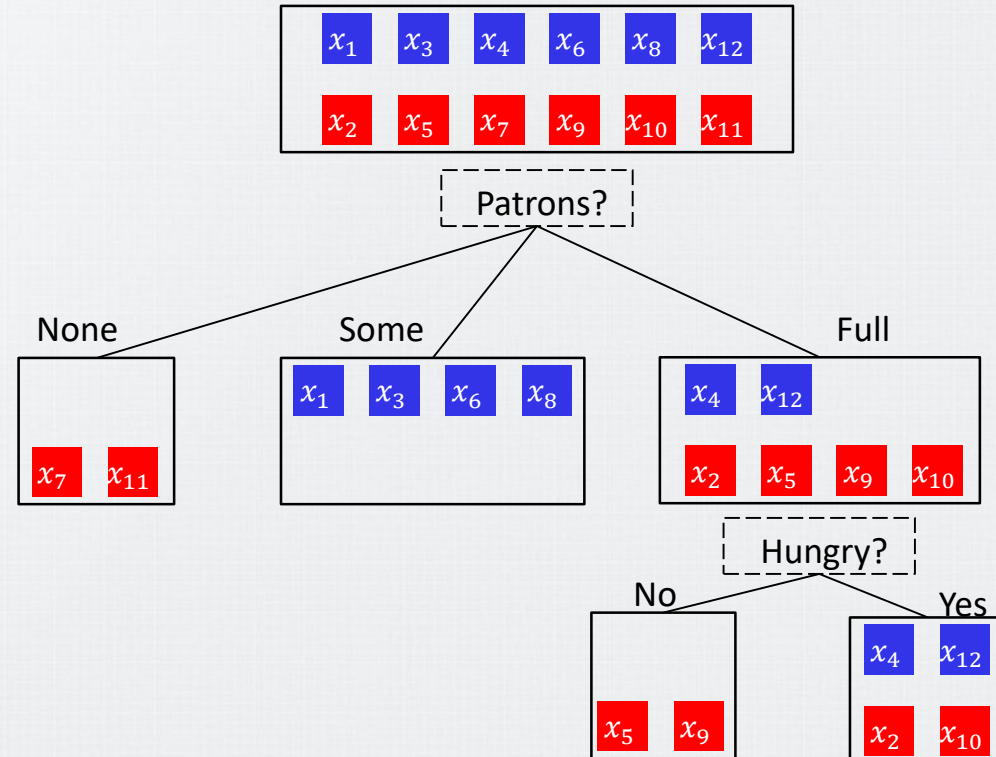
Features				y
Sample	...	Type	...	WillWait
x_1	...	French	...	Yes
x_2	...	Thai	...	No
x_3	...	Burger	...	Yes
x_4	...	Thai	...	Yes
x_5	...	French	...	No
x_6	...	Italian	...	Yes
x_7	...	Burger	...	No
x_8	...	Thai	...	Yes
x_9	...	Burger	...	No
x_{10}	...	Italian	...	No
x_{11}	...	Thai	...	No
x_{12}	...	Burger	...	Yes



Decision Trees for Categorical Data

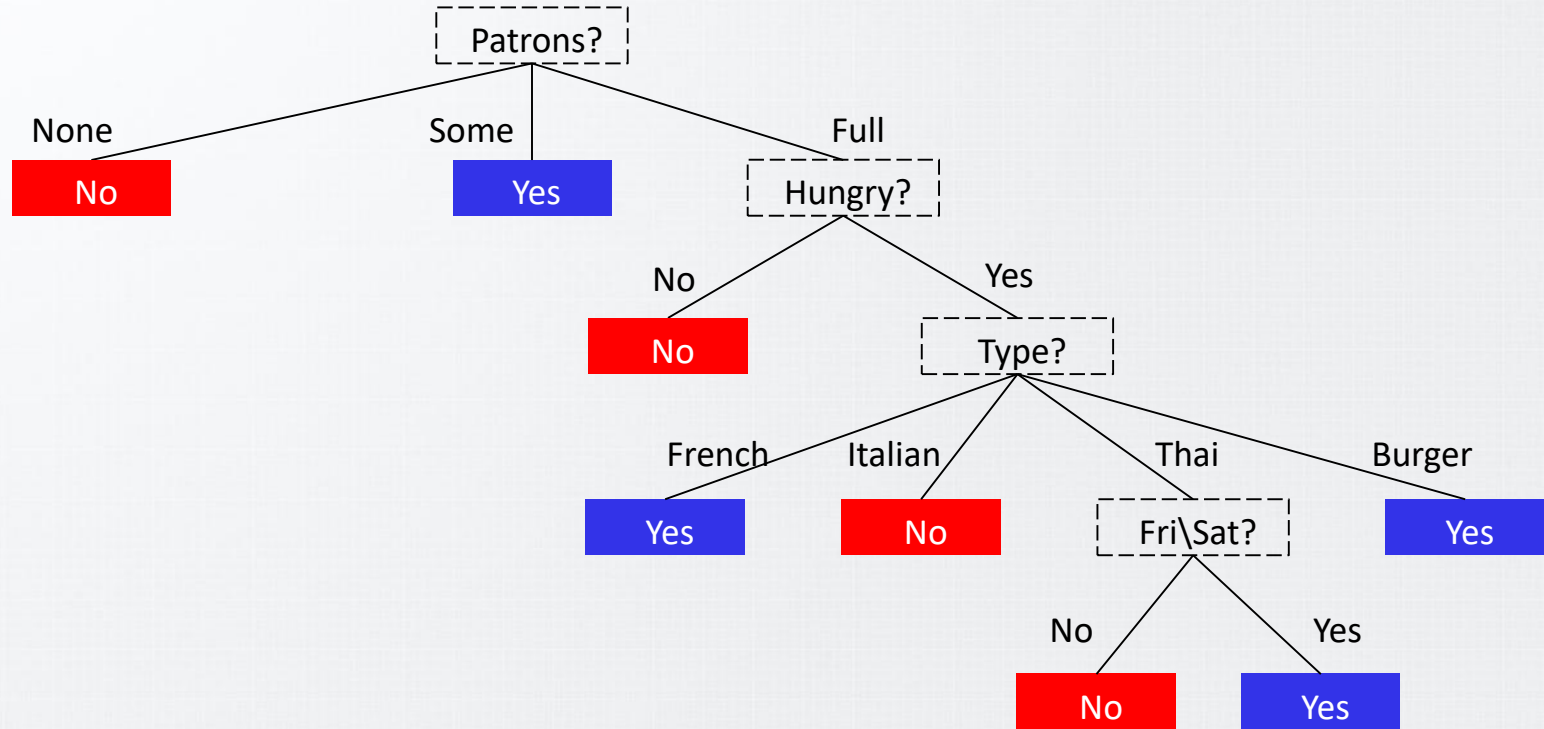
Decision Trees for Regression and Categorical Target Variables

Features				y
Sample	...	Pat	...	WillWait
x_1	...	Some	...	Yes
x_2	...	Full	...	No
x_3	...	Some	...	Yes
x_4	...	Full	...	Yes
x_5	...	Full	...	No
x_6	...	Some	...	Yes
x_7	...	None	...	No
x_8	...	Some	...	Yes
x_9	...	Full	...	No
x_{10}	...	Full	...	No
x_{11}	...	None	...	No
x_{12}	...	Full	...	Yes



Decision Trees for Categorical Data

Decision Trees for Regression and Categorical Target Variables



Types of Decision Tree

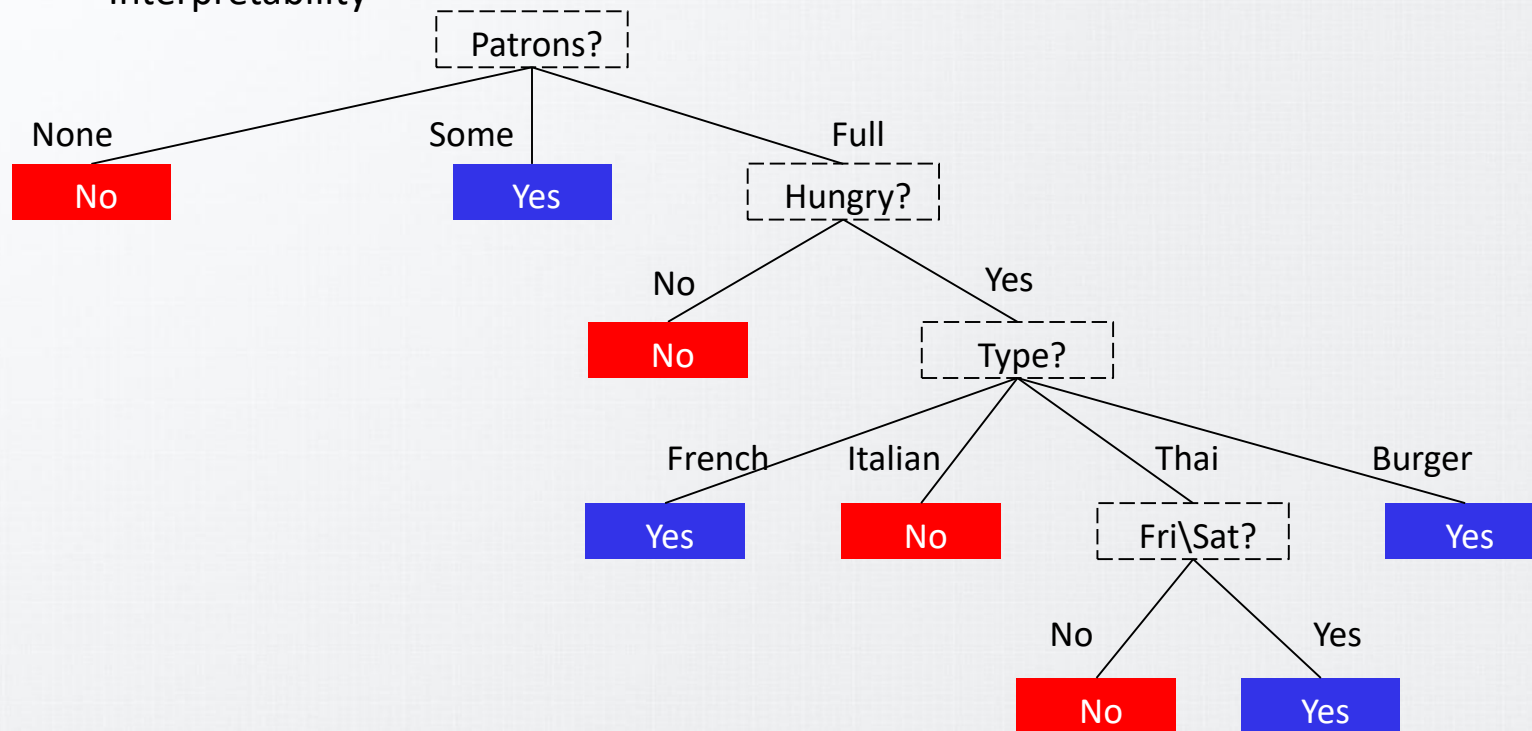
Decision Trees for Regression and Categorical Target Variables

Property	ID3	CART
Split Criterion	Entropy	Gini (classification), MSE (regression)
Node type	Multiway	Binary
Features	Mostly categorical	Continuous and categorical
Output	Classification	Classification and Regression
Split type	One branch per category	Binary split

Final Notes

Building a Decision Tree

- The model provides interpretable decisions
 - Interpretability



Final Notes

Building a Decision Tree

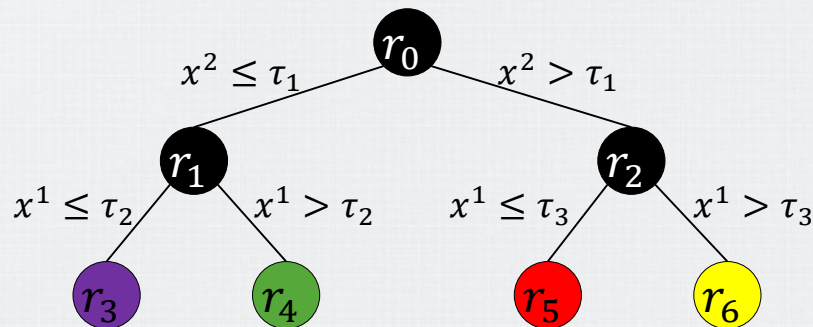
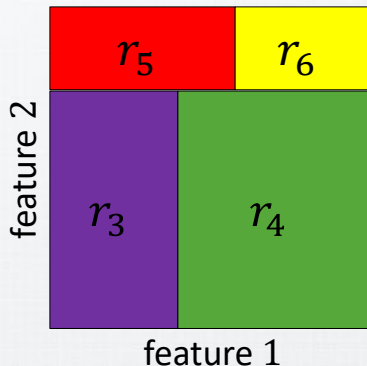
- Decision trees are versatile and adaptable to different problems
 - Categorical and numerical features
 - Classification and Regression
- When do we stop growing the decision tree?
 - Until the leaf nodes are pure (training error is 0)
 - Grow the tree to its maximum size, and then prune it back

Oblique Decision Trees

Introduction

Oblique Decision Trees

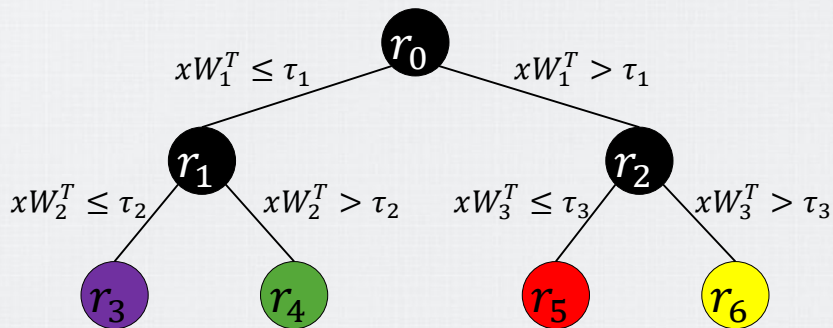
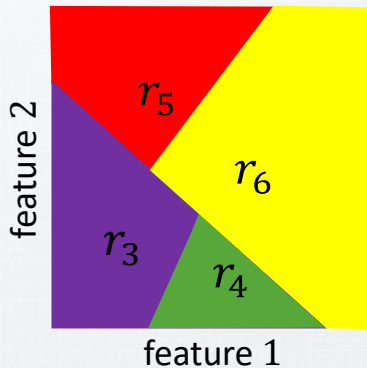
- The previous decision trees are fundamentally built on **axis-aligned** decisions
 - A node performs a test considering only **one dimension** at a time
- Limitations of axis-aligned trees
 - Require deeper trees when the decision boundary is not aligned with the feature axes



Oblique Trees

Oblique Decision Trees

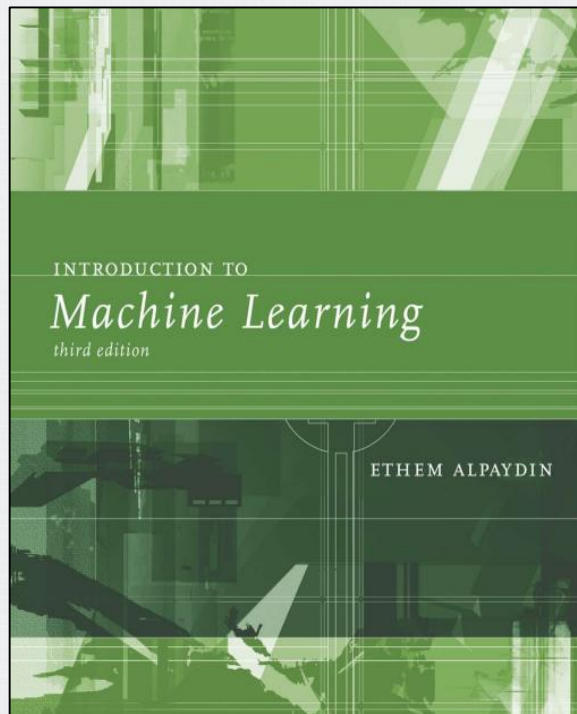
- Splits occur along hyperplanes of the form xW_i^T
 - W_i^T is a learned hyperplane at node i



Bibliography and Recommend Lecture

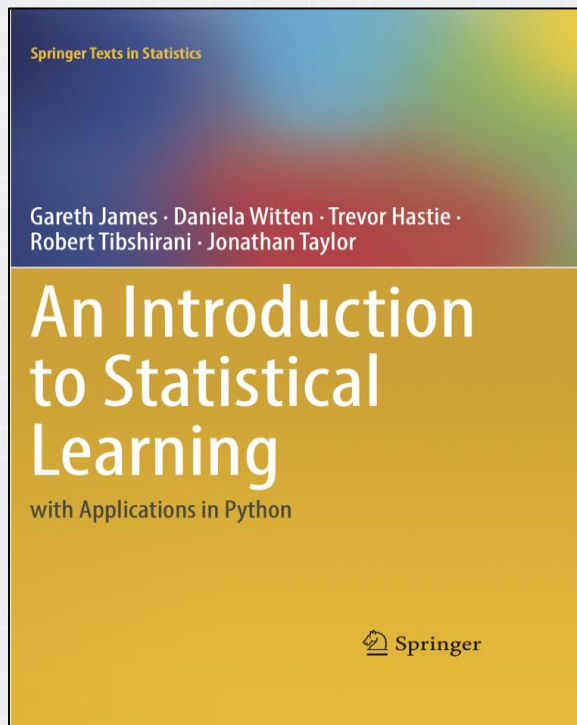
Bibliography

- INTRODUCTION TO Machine Learning
 - Chapter 9
 - Decision Trees
 - 9.6 Multivariate Trees



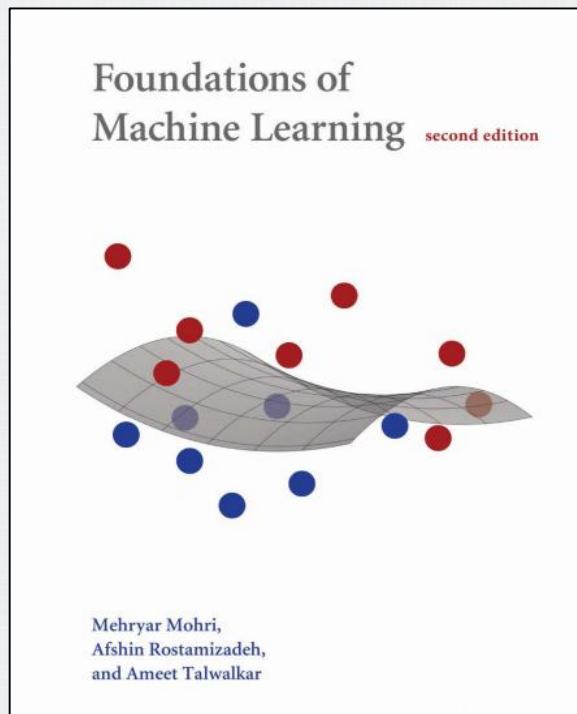
Bibliography

- An Introduction to Statistical Learning
 - Chapter 8
 - Tree-Based Methods



Bibliography

- Foundations of Machine Learning
 - Chapter 9
 - 9.3.3 Decision trees



Bibliography

- Sarwesh Rauniyar. *Jacobian Aligned Random Forests*. International Conference on Learning Representations (ICLR), 2026

